

Lab3 Extra

准备工作：创建并切换到 lab3-extra 分支

请在自动初始化分支后，在开发机依次执行以下命令：

```
$ cd ~/学号
$ git fetch
$ git checkout lab3-extra
```

初始化的 lab3-extra 分支基于课下完成的 lab3 分支，并且在 tests 目录下添加了 lab3-bp 样例测试目录。

问题描述

课下实验中，我们主要介绍了异常的分发、异常向量组和 0 号异常（时钟中断）的处理过程。在本次 Extra 中，我们希望大家拓展 9 号 bp 异常的异常分发，并在此基础上实现自定义的异常处理。

Bp 异常（Breakpoint）由 break 指令触发，在 MIPS 指令集中，对该异常的描述如下：

ExcCode	助记符	描述
9	Bp	执行 Break 指令

break 指令用于软件断点，其机器码格式如下：

31-26(op)	25-6(code)	5-0(funcnt)	格式
000000	code (20 bits)	001101	break code

其中 op 字段恒为 0，funcnt 字段恒为 0x0d，而 code 字段是一个 20 位的立即数，可用于传递不同的参数。

在本次题目中，我们要实现自定义的 Bp 异常处理，根据 code 的不同取值执行不同的操作：

- 若 code == 0x0，表示用户断点，直接退出异常处理，继续执行后续指令。
- 若 code == 0x6，数组访问越界触发，程序会将数组长度存入 \$a0（4号寄存器），要求对访存指令（当前 break 指令的下一条指令）的立即数(imm)部分进行检查和修改，对应索引为(imm >> 2)：
- 若越界的索引为负数，则将立即数设置为0。
- 若越界的索引大于等于数组长度，则修改立即数，使得索引为数组长度减1，即数组的最后一个元素。

越界检查格式如下：

```
... // 数组长度存入 $a0
break 6
lw/sw $t2, imm(base)
```

base 是数组首地址，imm 立即数是索引。

- 若 `code == 0x7`，除零异常触发，要求获取将要发生错误的除法指令（当前 `break` 指令的下一条指令），并且修改除数为2。

除零检查格式如下：

```
bne $9, $zero, div_ok
break 7
div_ok: div $8, $9
```

`div`，`lw`，`sw` 指令的机器码如下：

指令	31-26(op)	25-21	20-16	15-11	10-6	5-0(func)
div	000000	rs (被除数)	rt (除数)	00000	00000	011010

指令	31-26(op)	25-21	20-16	15-0
lw	100011	base	rt	imm
sw	101011	base	rt	imm

在异常处理完成，返回用户态之前，需要输出相应信息：

- 用户断点

```
printk("Break skip!\n");
```

- 越界检查

```
printk("Out of bounds handled, new imm is : %04x!\n", new_imm); // new_imm为替换后的立即数
```

- 除零检查

```
printk("Divide by zero handled!\n");
```

注意：在异常处理结束前，必须让 `EPC` 指向下一条指令，否则会反复触发 `Bp` 异常。如果你的异常处理程序运行超时，请检查该步骤是否实现正确。

题目要求

1. 建立 `Bp` 异常的处理函数。
2. 完成异常的分发，将 9 号异常绑定到该处理函数。
3. 在处理函数中，根据 `break` 指令中的 `code` 字段实现上述三种分支，并输出相应信息。

提示

1. 在 `kern/genex.S` 中，用 `BUILD_HANDLER` 宏来构建异常处理函数（切勿添加到 `if` 中）：

```

#if !defined(LAB) || LAB >= 4
BUILD_HANDLER mod do_tlb_mod
BUILD_HANDLER sys do_syscall
#endif

BUILD_HANDLER reserved do_reserved
BUILD_HANDLER bp do_bp

```

2. 在 `kern/traps.c` 修改异常向量组，并实现异常处理函数：

```

// 声明 handle 函数
extern void handle_bp(void);

// exception_handlers 请按以下代码实现
void (*exception_handlers[32])(void) = {
[0 ... 31] = handle_reserved,
[0] = handle_int,
[2 ... 3] = handle_tlb,
[9] = handle_bp,
#if !defined(LAB) || LAB >= 4
[1] = handle_mod,
[8] = handle_sys,
#endif
};

void do_bp(struct Trapframe *tf) {
/* 1. 获取 break 指令及其 code */
/* 2. 分类处理异常 */
switch (code) {
case 0x0:
// 处理用户断点
break;
case 0x6:
// 处理索引越界
break;
case 0x7:
// 处理除数为零
break;
}
return;
}

```

3. 在完成处理函数时：

- 本题涉及对寄存器值的读取和修改。你需要访问或修改保存现场的 `Trapframe` 结构体（定义在 `include/trap.h` 中）中对应通用寄存器。
- 本题涉及到对内存的访问。
- 可以通过 **EPC寄存器增减4** 获取当前指令的前后指令的虚拟地址。
- 由于我们要修改的指令部分在 `kuseg` 区间，这一部分的虚拟地址需要通过 TLB 来获取物理地址。我们设置了程序中保存操作指令代码的 `.text` 节权限为只读，这部分空间在页表中仅被映射为 `PTE_V`，而不带有 `PTE_D` 权限，因此经由页表项无法对物理页进行写操作。可以考虑查询 `curenv` 的页表获取对应指令的物理地址，再转化为 `kseg0` 的虚拟地址，从而修改相应内容。

- 因为本题中对内存访问较多，建议构造虚拟地址转化到内存中指令地址的功能辅助函数。

```
int *va2instrAddr(unsigned long va) {  
    /* 1. 查询 curenv 的页表获取虚拟地址对应页表项； */  
    /* 2. 通过页表项和虚拟地址得到物理地址； */  
    /* 3. 将该物理地址转化为 kseg0 区间中虚拟地址； */  
    /* 可能会使用到的函数和宏: page_lookup, PTE_ADDR, KADDR */  
}
```

- 可以通过更改 CP0 中 EPC 寄存器（对应 `Trapframe` 结构体中的成员 `cp0_epc`）的值，使得异常恢复后执行的是下一条指令。
4. MIPS 中，lw/sw 要求地址4字节对齐。从指令立即数提取字索引时，需右移2位恢复原值；以字为单位的数组索引访问时，需左移2位转换为字节偏移。立即数不保证为正数，你需要对16位立即数的符号位进行维护（例如使用16位有符号类型 short 存储立即数）。

题目约束

1. 测试程序 Bp 异常只会出现 code 为 0x0, 0x6 和 0x7 这三种情况，不会出现其他值。
2. 测试保证所有越界异常的 break 指令，base 寄存器一定不是4号寄存器，base 中存储的是数组首地址。不保证 lw 和 sw 指令的立即数为正数。
3. 测试保证所有 div 指令都会进行除零检测且只会因为除零陷入 Bp 异常，break 指令一定是除法指令的前一条指令。除法指令只会出现 div，且不会发生宏展开，不会发生除法溢出。
4. 本题保证发生 Bp 异常的指令的上一条指令不会是任意跳转指令，即保证发生 Bp 异常的指令不会出现在延迟槽中。

样例输出 & 本地测试

本地测试样例如下：

```
int main() {  
    unsigned int ret = 0;  
  
    // user break  
    _BUILD_BREAK_USER();  
  
    // oob break  
    int array[5] = {0, 1, 2, 3, 4};  
    int len = 5;  
    int dst = 5;  
    int index = -1;  
    _BUILD_OOB_BREAK_LW(dst, array, index, len);  
    if (dst == 5) {  
        ret |= BIT(0);  
    } else if (dst != 0) {  
        ret |= BIT(1);  
    }  
  
    // div0 break  
    int rs, rt;  
    rs = 0x1;  
    rt = 0x0;
```

```

_BUILD_BREAK_DIV0(rs, rt);
if (rt == 0) {
    ret |= BIT(2);
} else if (rt != 2) {
    ret |= BIT(3);
}

return ret;
}

```

你可以使用：

- `make test lab=3_bp && make run` 在本地测试上述样例（调试模式）
- `MOS_PROFILE=release make test lab=3_bp && make run` 在本地测试上述样例（开启优化）

运行正确的结果应当如下：

```

Break skip!
Out of bounds handled, new imm is : 0000!
Divide by zero handled!
[00000800] free env 00000800
i am killed ...

```

注：如果要本地构建样例测试，由于break的机器码在 GNU 中与 Mips 不同，请使用 `.word` (`<< 6) | 0x0d`) 代替 `break code` 进行样例构造。你也可以使用样例提供的宏定义进行测试

提交评测 & 评测标准

请在开发机中执行下列命令后，在课程网站上提交评测。

```

$ cd ~/学号
$ git add -A
$ git commit -m "message" # 请将 message 改为有意义的信息
$ git push

```

在线评测时，所有的 `.mk` 文件、所有的 `Makefile` 文件、`init/init.c` 以及 `tests/` 和 `tools/` 目录下的所有文件都可能被替换为标准版本，因此请同学们在本地开发时，**不要**在这些文件中编写实际功能所依赖的代码。

具体要求和分数分布如下：

测试点序号	测试点内容	测试点分值
1	样例	10分
2	仅有 <code>code = 0x0</code> 的 Bp异常	10分
3	仅有 <code>code = 0x6</code> 的 Bp异常	30分
4	仅有 <code>code = 0x7</code> 的 Bp异常	20分
5	多种 <code>code</code> 混合的 Bp异常	30分

测试点依赖关系如下（箭头方向表示依赖方向）：

